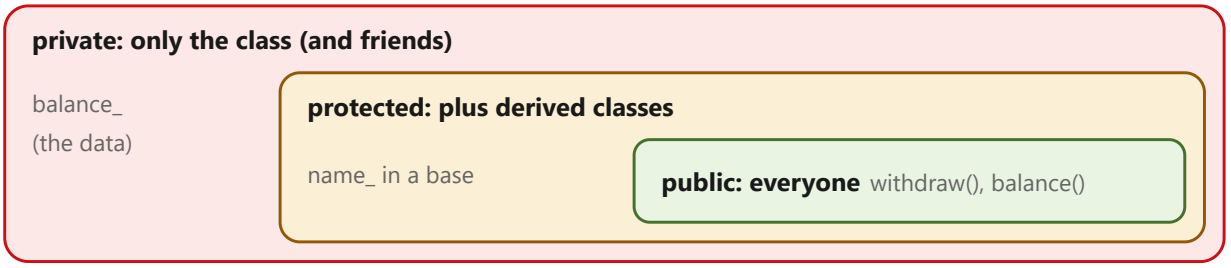
Classes and Objects

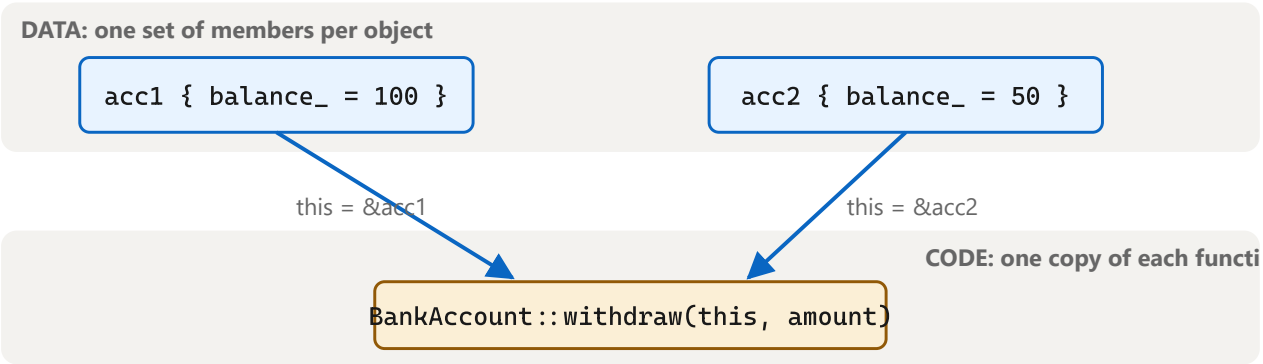
Classes and objects: encapsulation and invariants, member functions with `this`, `const` and `static`, and constructors and destructors. Assumes Pointers and References.

Classes and encapsulation



Hide the data, expose operations that keep the invariant true.

A **class** bundles data with the functions that use it. **Encapsulation**: keep data private, expose only operations that keep it valid.



Objects carry data; the code is shared. The hidden this pointer tells the function which object to work on.

```
class BankAccount {
// the interface other code may use
public:
    explicit BankAccount(double opening) : balance_(opening) {}

    bool withdraw(double amount) {
        // invariant protected
        if (amount ≤ 0 || amount > balance_) return false;
```

```
        balance_ -= amount;
        return true;
    }
    double balance() const { return balance_; }

private:                                    // the implementation, hidden
    double balance_;
};

BankAccount acc{100.0};
acc.withdraw(30);           // fine
// acc.balance_ = 1e9; // ERROR: private, nobody can break the invariant
```

SPECIFIER	WHO CAN ACCESS
`public`	everyone
`protected`	the class itself and classes derived from it
`private`	only the class itself (and its friends)

"Valid" is the class **invariant** (here: balance never negative). Hide the representation, let constructors build only valid objects, keep the public interface small. No invariant to protect? Use a plain `struct`.

> **STRUCT VS CLASS** Only the default access differs: `struct` public, `class` private. Use `struct` for plain data.

Member functions: `this`, `const`, `static`, `friend`

A member function is an ordinary function with one hidden extra parameter, `this`, the address of the object it was called on. `static` members have no `this`: they belong to the class.

```
class Counter {
    int n_ = 0;
    // ONE copy shared by all Counter objects
    static int instances_;
public:
    Counter() { ++instances_; }
```

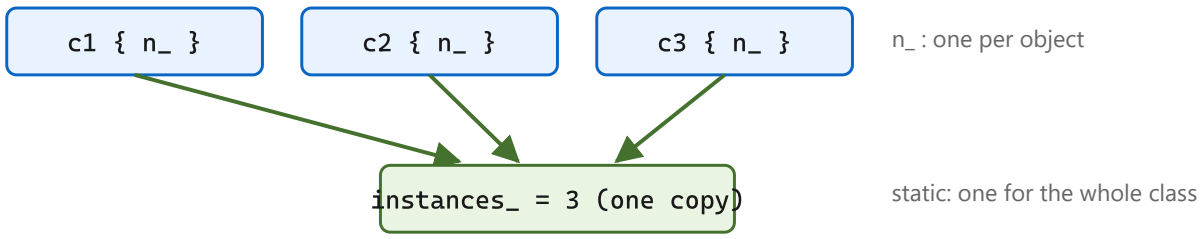
```
    ~Counter() { --instances_; }

    // `this` is a pointer to the current object
    Counter& add(int k) { n_ += k; return *this; }
    // const: promises not to modify the object
    int get() const { return n_; }
    // no `this`: callable as Counter::instances()
    static int instances() { return instances_; }

    // may touch private members
    friend std::ostream& operator<<(std::ostream&, const Counter&);
};
// a static data member must be defined once
int Counter::instances_ = 0;

Counter c;
// chaining works because add returns *this
c.add(1).add(2);
```

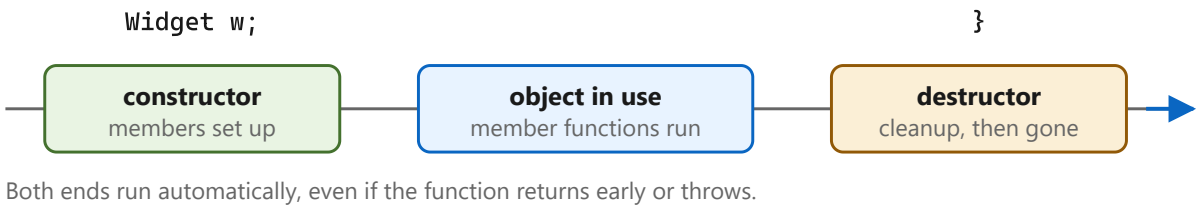
- `const` **member function**: callable on const objects, cannot modify members (except `mutable`). Make getters const.
- **static member function**: belongs to the class, no `this`.
- `friend`: access to privates. Use sparingly.



A static member belongs to the class, not to any object.

Constructors and destructors

A **constructor** sets an object up; the **destructor** (`~Name()`) runs automatically when its lifetime ends. This is the basis of RAII.



The constructor and the destructor bracket the life of every object.

```
class Widget {
    const int id_;
    std::string name_;
public:
    // delegating constructor
    Widget() : Widget(0) {}
    Widget(int id, std::string n = "none")
        // member initializer list
        : id_(id), name_(std::move(n)) {}
    ~Widget() { /* release resources here */ }
};
```

- **Use the initializer list**: it initializes members directly. `const` and reference members can only be set there.
- **Members initialize in declaration order**, not list order (`-Wreorder`).
- `explicit` on a one-argument constructor blocks silent conversions.

A first look at RAII

Because the destructor always runs when the object goes out of scope, a class can **acquire** a resource in its constructor and **release** it in its destructor. You never have to remember to clean up. This idea, **RAII**, has its own lesson later (RAII and exceptions); for now it is enough to see the shape of it:

```
class File {
    std::FILE* f_;
public:
    // acquire in the constructor
    explicit File(const char* path) : f_(std::fopen(path, "r")) {}
    // release in the destructor
    ~File() { if (f_) std::fclose(f_); }
    // see the next section: copying would close it twice
    File(const File&) = delete;
    File& operator=(const File&) = delete;
};

void work() {
    File f("data.txt");
    // use f ... a return or an exception below still runs ~File
}   // f is closed here, automatically
```

Order of construction and destruction

Construction: base, members, derived body. Destruction: the exact reverse.

> KEY POINT Construct: base, members, derived body. Destruct: reverse.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Classes and encapsulation	C++ Primer	7.1 Defining abstract data types	283 to 296
	C++ Primer	7.2 Access control and encapsulation	297 to 299
	C++ Primer	7.4 Class scope	311 to 316
	Programming: Principles and Practice	8.2, 8.3 Classes and members, interface and implementation	236 to 238
	Programming: Principles and Practice	8.7 Class interfaces	250 to 258
	Programming: Principles and Practice	12.3, 12.5 Base and derived classes; benefits of object-oriented programming	380 to 387 and 389 to 390
Member functions: this, const, static, friend	C++ Primer	7.1 Defining abstract data types	283 to 296
	C++ Primer	7.3 Additional class features	300 to 310
	C++ Primer	7.6 static class members	329 to 334
	Programming: Principles and Practice	15.6 to 15.8 Element access, an example list, the this pointer	464 to 473
	C++ Move Semantics	Chapter 5 Overloading on reference qualifiers	97 to 106
Constructors and destructors	C++ Primer	7.5 Constructors revisited	317 to 328
	C++ Primer	15.7 Constructors and copy control	651 to 658
	Programming: Principles and Practice	15.3 to 15.5 Memory, addresses and pointers; free store; destructors	452 to 463